

Giáo Án

Giới thiệu Context API

1. Mục tiêu

- Học viên nhận diện và hiểu rõ vấn đề "Props Drilling" trong React.
- Học viên hiểu được mục đích ra đời và lợi ích của Context API trong việc giải quyết vấn đề Props Drilling.
- Học viên nắm bắt được khái niệm cơ bản về Context và vai trò của nó.

2. Nội dung bài giảng

2.1 Khởi động và đặt vấn đề:

- Bắt đầu bằng cách đặt câu hỏi cho học viên về cách truyền dữ liệu giữa các component trong React hiện tại (Props).
- Vẽ một sơ đồ cây component đơn giản lên bảng hoặc trình chiếu. Ví dụ: `App → Layout → ProductList → ProductItem → AddToCartButton`.
- Đặt tình huống:** Chúng ta muốn hiển thị **số lượng sản phẩm trong giỏ hàng** ngay trên nút `AddToCartButton` (ví dụ: "Thêm vào giỏ (3)"). Nhưng **Dữ liệu giỏ hàng** (số lượng sản phẩm trong giỏ) được đặt ở component `App` (ví dụ, dùng `useState`).
- Giảng viên:** Hỏi học viên cách truyền dữ liệu số lượng sản phẩm trong giỏ hàng này đến `AddToCartButton`.
- Giảng viên:** Thảo luận về việc sử dụng props và minh họa luồng props `cartItemCount` đi qua `Layout`, `ProductList`, `ProductItem`, dù các component này không thực sự cần dữ liệu số lượng giỏ hàng cho mục đích hiển thị hay logic của riêng chúng. **Đây chính là Props Drilling.**
- Giảng viên:** Đặt câu hỏi: "Điều gì xảy ra nếu cây component sâu hơn nữa hoặc có nhiều component cần thông tin giỏ hàng? Việc props drilling sẽ trở nên phức tạp và khó quản lý như thế nào?"

2.2 Giới thiệu Context API

- Giới thiệu Context API như một "giải pháp" cho vấn đề Props Drilling.
- Giải thích:
 - Context API là gì?** Một cơ chế của React cho phép chia sẻ dữ liệu "toàn cục" trong một phạm vi nhất định của cây component mà không cần truyền props thủ công qua từng cấp.
 - Ví dụ so sánh:** Tưởng tượng Context như một "nguồn dữ liệu chung" hoặc "trung tâm dữ liệu" được thiết lập ở một vị trí nào đó trong cây component. Các component "con cháu" trong phạm vi đó có thể trực tiếp truy cập và sử dụng dữ liệu này mà không cần qua trung gian (props).
 - Nhấn mạnh:** Context API **không phải** là thay thế cho các thư viện quản lý state toàn cục phức tạp như Redux hay Zustand. Nó phù hợp hơn cho việc chia sẻ dữ liệu ở mức độ "ứng dụng con" hoặc "module", hoặc cho các dữ liệu mang tính chất theme, locale, user session,...
- Liệt kê lợi ích của Context API:
 - Giải quyết Props Drilling:** Code gọn gàng, dễ đọc, dễ bảo trì hơn.
 - Tái sử dụng component:** Component "con cháu" trở nên độc lập hơn, không phụ thuộc vào việc props được truyền từ đâu. `AddToCartButton` không cần biết `cartItemCount` đến từ component cha nào.
- Giới thiệu sơ các thành phần chính của Context API:
 - `createContext`: Hàm để tạo ra một đối tượng Context (ví dụ: `CartContext`).
 - `Provider`: Component để "cung cấp" dữ liệu Context (ví dụ: `<CartContext.Provider>`) cho các component con cháu.
 - `Consumer` (hoặc `useContext` Hook): Component để "tiêu thụ" hoặc "đọc" dữ liệu Context (ví dụ: `<CartContext.Consumer>` hoặc `useContext(CartContext)`).
- 📌 Lưu ý cho giảng viên:

- **Sử dụng ví dụ Shopping Cart để minh họa rõ ràng vấn đề "Props Drilling"** và làm cho học viên cảm thấy vấn đề này thực tế và cần giải quyết.
- **Giải thích rõ ràng mục đích của Context API:** không phải là state management toàn cục, mà là giải pháp cho chia sẻ dữ liệu cục bộ và Props Drilling, đặc biệt trong ngữ cảnh dữ liệu giỏ hàng, theme, cài đặt ứng dụng....
- 🖱 Cho học viên làm Bài 1 - 5 trong file **Bài Tập Cuối Chương**.

Thiết lập và thao tác với Context API

1. Mục tiêu

- Học viên nắm vững các bước để thiết lập Context API trong ứng dụng React.
- Học viên biết cách sử dụng createContext, Provider, Consumer và useContext để chia sẻ và truy cập dữ liệu Context.
- Học viên hiểu cơ chế cập nhật dữ liệu Context và ảnh hưởng của nó đến các component tiêu thụ.

2. Nội dung bài giảng

2.1 createContext()

- `createContext()` tạo ra một **đối tượng Context**.
- Giải thích đối tượng Context này chứa: `Provider`, `Consumer`, `displayName`.
- Thảo luận về giá trị mặc định (default value) của `createContext()`.
- Ví dụ code tạo Context cho **giỏ hàng** (Cart Context) - đơn giản ban đầu có thể chỉ cần quản lý `cartItemCount`:

```
const CartContext = createContext({
  cartItemCount: 0, // Số lượng sản phẩm mặc định trong giỏ là 0
  setCartItemCount: () => {} // Hàm mặc định (sẽ được override bởi Provider)
});
```

2.2 Context.Provider

- Giải thích vai trò **bắt buộc** của Provider.
- Nhấn mạnh **thuộc tính value** của Provider.
- value có thể là **bất kỳ giá trị JavaScript nào**.
- Ví dụ code sử dụng `CartContext.Provider` trong App component:

```
function App() {
  const [cartItemCount, setCartItemCount] = useState(0);
  const cartValue = { cartItemCount, setCartItemCount };

  return (
    <CartContext.Provider value={cartValue}>
      <Layout>
        <Header />
        <ProductList />
      </Layout>
    </CartContext.Provider>
  );
}
```

2.3 Tiêu thụ Context chi tiết

- **Context.Consumer:**
 - Giải thích `Consumer` là component **render props**.

- Giải thích Cú pháp và cách hoạt động của `Consumer` .
- Ví dụ code `AddToCartButton` sử dụng `Consumer` để hiển thị số lượng giỏ hàng:

```
function AddToCartButton() {
  return (
    <CartContext.Consumer>
      {{{ cartItemCount }}} => (
        <button>
          Thêm vào giỏ {{{cartItemCount}}}
        </button>
      )
    </CartContext.Consumer>
  );
}
```

- **Nhược điểm của Consumer:** Cú pháp hơi dài dòng, lồng ghép, khó đọc khi có nhiều Context hoặc logic phức tạp.

- **useContext Hook:**

- Giới thiệu `useContext` Hook là cách **đơn giản và hiện đại hơn** để tiêu thụ Context trong **functional component**.
- Cú pháp: `const value = useContext(MyContext);` .
- **Giảng viên:** `useContext(MyContext)` sẽ trả về giá trị Context được cung cấp bởi `Provider` **gần nhất** của `MyContext` .
- Ví dụ code `CartIcon` (icon giỏ hàng trên header) sử dụng `useContext` để hiển thị số lượng giỏ hàng:

```
function CartIcon() {
  const { cartItemCount } = useContext(CartContext);

  return (
    <div>
      🛒 {{{cartItemCount}}}
    </div>
  );
}
```

- **Ưu điểm của useContext:** Cú pháp ngắn gọn, dễ đọc, dễ sử dụng trong functional component, giống như lấy một biến thông thường.

- 📌 **Lưu ý cho giảng viên:**

- **Nhấn mạnh rằng useContext là cách tiếp cận hiện đại và thường được ưu tiên hơn** trong functional component vì cú pháp đơn giản và dễ đọc.
- **Consumer vẫn có giá trị trong một số trường hợp cụ thể** hoặc khi làm việc với codebase cũ, và hiểu về Consumer giúp học viên đọc code dễ dàng hơn.
- 📌 Cho học viên làm Bài 6 - 9 trong file **Bài Tập Cuối Chương**.

Giới thiệu useRef và cách sử dụng để tránh re-render dư thừa

1. Mục tiêu:

- Học viên hiểu được khái niệm cơ bản về `useRef` Hook và mục đích sử dụng chính của nó.
- Học viên biết cách tạo và truy cập `ref` bằng `useRef` .
- Học viên nắm được các use case đơn giản và trực quan của `useRef` trong React.

2. Nội dung bài giảng

2.1 Đặt vấn đề và giới thiệu `useRef`

- Bắt đầu bằng cách ôn lại `useState` Hook và cách nó quản lý state, gây re-render khi state thay đổi.
- Đặt câu hỏi: "Trong React, có khi nào chúng ta muốn **lưu trữ một giá trị** trong component, nhưng **không muốn nó là state** → đó là khi **không muốn component re-render** khi giá trị đó thay đổi không?"
- Giới thiệu `useRef` **Hook** như một công cụ để giải quyết vấn đề này.
- Giải thích:
 - `useRef` **là gì?** Hook tạo ra một **"reference" (tham chiếu)**, giống như một "chiếc hộp" để bạn lưu trữ bất kỳ giá trị nào. Bạn có thể thay đổi giá trị bên trong "chiếc hộp" này, nhưng React sẽ **không re-render** component khi bạn làm điều đó.
 - `useRef()` **trả về một object:** Object này có thuộc tính `.current`. Giá trị khởi tạo được truyền vào `useRef(initialValue)` sẽ được gán cho thuộc tính `.current`.
 - **Ví dụ so sánh trực quan:** Tưởng tượng `useState` như một "công tắc đèn" - khi bạn bật/tắt công tắc (thay đổi state), đèn (UI) sẽ thay đổi theo (re-render).
Còn `useRef` giống như một "mẫu giấy nháp" - bạn có thể viết/xóa bất cứ thứ gì trên giấy (thay đổi `.current`) mà không ảnh hưởng đến đèn (UI) cả.

2.2 Ví dụ 1: Bộ đếm ẩn

```
import React, { useRef } from 'react';

function HiddenCounter() {
  const counterRef = useRef(0); // Tạo ref để lưu trữ bộ đếm

  const incrementCounter = () => {
    counterRef.current++; // Tăng giá trị ref
    console.log('Bộ đếm:', counterRef.current);
  };

  return (
    <div>
      <button onClick={incrementCounter}>Tăng bộ đếm ẩn</button>
      { /* Không hiển thị counterRef.current trên UI */ }
    </div>
  );
}

export default HiddenCounter;
```

- 📌 **Lưu ý cho giảng viên:** Chạy ví dụ và giải thích:
 - `counterRef` được tạo bằng `useRef(0)` và khởi tạo giá trị là 0.
 - Khi click button, `counterRef.current++` tăng giá trị của ref.
 - `console.log` hiển thị giá trị bộ đếm tăng lên.
 - **Nhấn mạnh:** Component **không hề re-render** khi `counterRef.current` thay đổi. Đây là điểm khác biệt chính với `useState`.
 - **Ứng dụng thực tế:** Bộ đếm ẩn có thể dùng cho nhiều mục đích logic bên trong component mà không cần hiển thị trên UI, ví dụ: đếm số lần một function được gọi, theo dõi trạng thái bên trong component.

2.3 Ví dụ 2: Truy cập DOM element trực tiếp

- Đặt tình huống: "Chúng ta muốn **focus vào một input field khi component vừa được hiển thị** (mount). Làm sao để làm được điều này trong React?"
- Giải thích: React thường không khuyến khích thao tác với **DOM thật** mà nên **thông qua React Virtual DOM (DOM ảo)**.
N

hưng trong một số trường hợp, truy cập trực tiếp DOM thật là cần thiết (ví dụ: focus, đo kích thước, gọi các method DOM API).

`useRef` cho phép chúng ta tạo tham chiếu đến các phần tử DOM thật.

- Code ví dụ **focus input khi mount**:

```
import React, { useRef, useEffect } from 'react';

function FocusInput() {
  const inputRef = useRef(null); // Khởi tạo ref, ban đầu không trỏ đến DOM element nào

  useEffect(() => {
    inputRef.current.focus(); // Gọi method focus() trên DOM element
  }, []); // useEffect chạy một lần sau khi component mount

  return (
    <div>
      <input ref={inputRef} type="text" placeholder="Nhập gì đó..." />
    </div>
  );
}

export default FocusInput;
```

- 📌 **Lưu ý cho giảng viên:** Chạy ví dụ và giải thích:
 - `inputRef = useRef(null)` : Tạo ref, ban đầu gán `null` vì chưa có DOM element nào.
 - `<input ref={inputRef} ... />` : Gán `ref` prop cho input element. React sẽ tự động gán DOM element tương ứng cho `inputRef.current` khi component mount.
 - `useEffect(() => { ... }, [])` : `useEffect` chạy một lần sau render đầu tiên (component mount).
 - `inputRef.current.focus()` : Truy cập DOM element thông qua `inputRef.current` và gọi method `focus()` để focus vào input.
 - **Chạy ví dụ:** Khi component `FocusInput` được render, input field sẽ tự động được focus.
 - **Nhấn mạnh:** `useRef` giúp chúng ta lấy được tham chiếu đến DOM element và thao tác với nó trực tiếp khi cần.
 - 🖱️ Cho học viên làm Bài 10 - 11 trong file **Bài Tập Cuối Chương**.